Collected C++ Files

RTP.cpp

```
#include "VantorBackend.h"
#include "quarantine.h"
#include "mlinference.h"

#include <windows.h>
#include <evntrace.h>
#include <tdh.h>
#include <psapi.h>

#include <string>
#include <vector>
#include <thread>
#include <future>
#include <algorithm>
#include <iostream>

#pragma comment(lib, "Advapi32.lib")
#pragma comment(lib, "Tdh.lib")
#pragma comment(lib, "Psapi.lib")

// ============================================================
// GLOBALS (Process RTP)
// ============================================================

static SafeDB* g_safe = nullptr;
static MLInference* g_ml = nullptr;
static ScanHistoryDB* g_history = nullptr;
static DWORD g_self_pid = 0;

// Microsoft-Windows-Kernel-Process
static const GUID KERNEL_PROCESS_PROVIDER =
{ 0x22FB2CD6, 0x0E7B, 0x422B,{0xA0,0xC7,0x2F,0xAD,0x1F,0xD0,0xE7,0x16}
};

// ============================================================
// UTF HELPERS
// ============================================================

static std::string utf16_to_utf8(const std::wstring& wstr) {
    if (wstr.empty()) return "";
    int size = WideCharToMultiByte(CP_UTF8, 0, wstr.c_str(), -1,
nullptr, 0, nullptr, nullptr);
    std::string str(size, 0);
```

```
    WideCharToMultiByte(CP_UTF8, 0, wstr.c_str(), -1, &str[0], size,
nullptr, nullptr);
    str.pop_back();
    return str;
}

// ============================================================
// DOCUMENT SANITIZATION
// ============================================================

static void sanitize_document(const std::string& path) {
    std::wstring wpath(path.begin(), path.end());
    std::wstring ads = wpath + L":Zone.Identifier";
    DeleteFileW(ads.c_str());
}

// ============================================================
// FILE DROP / BURST HANDLER
// ============================================================

static void process_event(
    std::string path,
    SafeDB& safe,
    const MLInference& ml,
    ScanHistoryDB& history
) {
    std::string ext;
    size_t dot = path.find_last_of(".");
    if (dot != std::string::npos)
        ext = path.substr(dot);

    std::transform(ext.begin(), ext.end(), ext.begin(), ::tolower);

    std::this_thread::sleep_for(std::chrono::milliseconds(600));

    if (ext == ".exe" || ext == ".bat" || ext == ".vbs" ||
        ext == ".ps1" || ext == ".scr" || ext == ".msi" || ext ==
".dll")
    {
        int sid = create_scan_session(history, "rtp", "RTP-FS");
        ScanResult r = scan_file(path, safe, ml, history, sid, "RTP-
FS");

        if (r.is_malicious) {
            handle_malicious_file(path, "RTP-FS", r.sha256, r.sha512);
        }
    }
    else if (ext == ".doc" || ext == ".docx" || ext == ".pdf" ||
        ext == ".xlsx" || ext == ".pptx")
```

```
    {
        sanitize_document(path);
    }
}

// ====================================================
// FILE SYSTEM RTP ENGINE
// ====================================================

static void start_filesystem_rtp(
    SafeDB& safe,
    const MLInference& ml,
    ScanHistoryDB& history
) {
    std::vector<std::wstring> targets;

    wchar_t* profile = nullptr;
    size_t len = 0;
    if (_wdupenv_s(&profile, &len, L"USERPROFILE") == 0 && profile) {
        targets.emplace_back(std::wstring(profile) + L"\\Downloads");
        targets.emplace_back(std::wstring(profile) + L"\\Desktop");
        free(profile);
    }

    for (const auto& dir : targets) {
        std::thread([dir, &safe, &ml, &history]() {
            HANDLE hDir = CreateFileW(
                dir.c_str(),
                FILE_LIST_DIRECTORY,
                FILE_SHARE_READ | FILE_SHARE_WRITE | FILE_SHARE_DELETE,
                nullptr,
                OPEN_EXISTING,
                FILE_FLAG_BACKUP_SEMANTICS,
                nullptr
            );

            if (hDir == INVALID_HANDLE_VALUE)
                return;

            BYTE buffer[4096];
            DWORD bytes;

            while (ReadDirectoryChangesW(
                hDir,
                buffer,
                sizeof(buffer),
                TRUE,
                FILE_NOTIFY_CHANGE_FILE_NAME |
FILE_NOTIFY_CHANGE_LAST_WRITE,
```

```
                &bytes,
                nullptr,
                nullptr))
            {
                auto* info =
reinterpret_cast<FILE_NOTIFY_INFORMATION*>(buffer);
                while (info) {
                    if (info->Action == FILE_ACTION_ADDED ||
                        info->Action == FILE_ACTION_RENAMED_NEW_NAME)
                    {
                        std::wstring fname(
                            info->FileName,
                            info->FileNameLength / sizeof(wchar_t)
                        );
                        std::string full =
                            utf16_to_utf8(dir + L"\\" + fname);

                        std::thread(
                            process_event,
                            full,
                            std::ref(safe),
                            std::ref(ml),
                            std::ref(history)
                        ).detach();
                    }

                    if (!info->NextEntryOffset) break;
                    info = reinterpret_cast<FILE_NOTIFY_INFORMATION*>(
                        reinterpret_cast<BYTE*>(info) + info-
>NextEntryOffset
                        );
                }
            }
            CloseHandle(hDir);
            }).detach();
    }
}

// ====================================================
// ETW PROCESS IMAGE EXTRACTION
// ====================================================
static bool extract_image_path(EVENT_RECORD* rec, std::wstring& out) {
    DWORD infoSize = 0;
    if (TdhGetEventInformation(rec, 0, nullptr, nullptr, &infoSize)
        != ERROR_INSUFFICIENT_BUFFER)
        return false;

    std::vector<BYTE> infoBuf(infoSize);
    auto info = reinterpret_cast<PTRACE_EVENT_INFO>(infoBuf.data());
```

```
if (TdhGetEventInformation(rec, 0, nullptr, info, &infoSize)
    != ERROR_SUCCESS)
    return false;

for (ULONG i = 0; i < info->TopLevelPropertyCount; i++) {
    auto& propInfo = info->EventPropertyInfoArray[i];

    PWSTR propName = reinterpret_cast<PWSTR>(
        reinterpret_cast<PBYTE>(info) + propInfo.NameOffset
        );

    if (wcscmp(propName, L"ImageFileName") != 0)
        continue;

    PROPERTY_DATA_DESCRIPTOR desc = {};
    desc.PropertyName = reinterpret_cast<ULONGLONG>(propName);
    desc.ArrayIndex = ULONG_MAX;

    // Query required size
    ULONG dataSize = 0;
    ULONG status = TdhGetProperty(
        rec,
        0,
        nullptr,
        1,
        &desc,
        0,
        nullptr
    );

    if (status != ERROR_INSUFFICIENT_BUFFER)
        return false;

    std::vector<BYTE> data(dataSize);

    // Fetch property
    if (TdhGetProperty(
        rec,
        0,
        nullptr,
        1,
        &desc,
        dataSize,
        data.data()
    ) != ERROR_SUCCESS)
        return false;

    out.assign(reinterpret_cast<PWSTR>(data.data()));
```

```
        return true;
    }

    return false;
}



// ========================================================
// ETW PROCESS CALLBACK
// ========================================================

static void WINAPI on_process_event(EVENT_RECORD* rec) {
    if (!rec) return;
    if (rec->EventHeader.EventDescriptor.Id != 1) return;

    DWORD pid = rec->EventHeader.ProcessId;
    if (pid == 0 || pid == g_self_pid) return;

    std::wstring image;
    if (!extract_image_path(rec, image)) return;

    std::string exe = utf16_to_utf8(image);

    std::cout << "[RTP] Process start PID=" << pid
        << " EXE=" << exe << "\n";

    std::this_thread::sleep_for(std::chrono::milliseconds(500));

    int sid = create_scan_session(*g_history, "rtp", "RTP-PROC");
    ScanResult r = scan_file(
        exe,
        *g_safe,
        *g_ml,
        *g_history,
        sid,
        "RTP-PROC"
    );

    if (r.is_malicious) {
        HANDLE h = OpenProcess(PROCESS_TERMINATE, FALSE, pid);
        if (h) {
            TerminateProcess(h, 1);
            CloseHandle(h);
        }
        handle_malicious_file(exe, "RTP-PROC", r.sha256, r.sha512);
    }
}
```

```
// ========================================================
// PROCESS RTP ENGINE (BLOCKING)
// ========================================================

static void start_process_rtp() {
    EVENT_TRACE_LOGFILE trace = {};
    trace.LoggerName = const_cast<LPWSTR>(L"RTP_PROCESS");
    trace.ProcessTraceMode =
        PROCESS_TRACE_MODE_REAL_TIME |
        PROCESS_TRACE_MODE_EVENT_RECORD;
    trace.EventRecordCallback = on_process_event;

    TRACEHANDLE h = OpenTrace(&trace);
    if (h == INVALID_PROCESSTRACE_HANDLE) {
        std::cerr << "[RTP] Failed to open process ETW trace\n";
        return;
    }

    ProcessTrace(&h, 1, nullptr, nullptr);
    CloseTrace(h);
}

// ========================================================
// PUBLIC ENTRY POINT
// ========================================================

void run_rtp_engine(
    SafeDB& safe,
    const MLInference& ml,
    ScanHistoryDB& history
) {
    g_safe = &safe;
    g_ml = const_cast<MLInference*>(&ml);
    g_history = &history;
    g_self_pid = GetCurrentProcessId();

    std::cout << "[RTP] Real-Time Protection ACTIVE\n";

    start_filesystem_rtp(safe, ml, history);

    start_process_rtp();
}
```

VantorBackend.cpp

```
#include "feature_extractor.h"
#include "heuristic.h"
#include "quarantine.h"
```

```
#include "signatures.h"
#include "VantorBackend.h"
#include "mlinference.h"
#include <fstream>
#include <filesystem>
#include <chrono>
#include <iomanip>
#include <sstream>
#include <openssl/evp.h>
#include <mutex>
#include <thread>

using ordered_json = nlohmann::ordered_json;
namespace fs = std::filesystem;

/* ========================   Global JSONL Logger
======================== */
static std::mutex g_log_mutex;
static const char* LOG_FILE = "logs.jsonl";

// Standard Logger (Fixed Schema for File Scans)
void log_event_json(const std::string& sid, const std::string&
timestamp, const std::string& path, const std::string& verdict) {
    std::lock_guard<std::mutex> lock(g_log_mutex);
    std::ofstream ofs(LOG_FILE, std::ios::app);
    if (!ofs) return;

    nlohmann::ordered_json log_obj;
    log_obj["SID"] = sid;
    log_obj["timestamp"] = timestamp;
    log_obj["file path"] = path;
    log_obj["verdict"] = verdict;

    ofs << log_obj.dump() << '\n';
}

// FIXED: Flexible Logger (Preserves ALL fields for ML/System events)
void log_event_json(const nlohmann::json& obj) {
    std::lock_guard<std::mutex> lock(g_log_mutex);
    std::ofstream ofs(LOG_FILE, std::ios::app);
    if (!ofs) return;

    auto tp = std::chrono::system_clock::now();
    std::time_t t = std::chrono::system_clock::to_time_t(tp);
    std::tm tm{};
    gmtime_s(&tm, &t);
    std::ostringstream oss;
    oss << std::put_time(&tm, "%Y-%m-%dT%H:%M:%SZ");
```

```
    // Copy the input object to an ordered_json to keep it readable
    nlohmann::ordered_json final_log = obj;

    // Ensure standard fields exist if they weren't provided
    if (!final_log.contains("SID")) {
        final_log["SID"] = "SYS-0000";
    }
    if (!final_log.contains("timestamp")) {
        final_log["timestamp"] = oss.str();
    }
    if (!final_log.contains("file path") && final_log.contains("path"))
{
        // Map "path" to "file path" for consistency, or keep both
        final_log["file path"] = final_log["path"];
    }

    // Write the FULL object, preserving scores and indices
    ofs << final_log.dump() << '\n';
}

/* ========================   Helpers   ======================== */
static std::string now_utc() {
    auto tp = std::chrono::system_clock::now();
    std::time_t t = std::chrono::system_clock::to_time_t(tp);
    std::tm tm{};
    gmtime_s(&tm, &t);
    std::ostringstream oss;
    oss << std::put_time(&tm, "%Y-%m-%dT%H:%M:%SZ");
    return oss.str();
}

bool compute_hashes(const std::string& path, std::string& sha256_out,
std::string& sha512_out) {
    auto digest = [](const std::string& p, const EVP_MD* md) ->
std::string {
        std::ifstream f(p, std::ios::binary);
        if (!f) return "";
        EVP_MD_CTX* ctx = EVP_MD_CTX_new();
        EVP_DigestInit_ex(ctx, md, nullptr);
        auto buf = std::make_unique<char[]>(16384);
        while (f.read(buf.get(), 16384) || f.gcount() > 0) {
            EVP_DigestUpdate(ctx, buf.get(),
static_cast<size_t>(f.gcount()));
        }
        unsigned char res[EVP_MAX_MD_SIZE];
        unsigned int len = 0;
        EVP_DigestFinal_ex(ctx, res, &len);
        EVP_MD_CTX_free(ctx);
        std::ostringstream ss;
```

```
        ss << std::hex << std::setfill('0');
        for (unsigned int i = 0; i < len; i++) ss << std::setw(2) <<
(int)res[i];
        return ss.str();
        };
    sha256_out = digest(path, EVP_sha256());
    sha512_out = digest(path, EVP_sha512());
    return !sha256_out.empty();
}

/* ========================   Safe DB   ======================== */
bool safe_db_init(SafeDB& safe_db, const std::string& path) {
    if (sqlite3_open(path.c_str(), &safe_db.db) != SQLITE_OK) return
false;
    const char* sql = "CREATE TABLE IF NOT EXISTS safe_files (sha512
TEXT PRIMARY KEY);";
    return sqlite3_exec(safe_db.db, sql, nullptr, nullptr, nullptr) ==
SQLITE_OK;
}

bool safe_db_contains(SafeDB& safe_db, const std::string& sha512) {
    sqlite3_stmt* stmt;
    const char* sql = "SELECT 1 FROM safe_files WHERE sha512=? LIMIT
1;";
    if (sqlite3_prepare_v2(safe_db.db, sql, -1, &stmt, nullptr) !=
SQLITE_OK) return false;
    sqlite3_bind_text(stmt, 1, sha512.c_str(), -1, SQLITE_STATIC);
    bool found = (sqlite3_step(stmt) == SQLITE_ROW);
    sqlite3_finalize(stmt);
    return found;
}

void safe_db_mark(SafeDB& safe_db, const std::string& sha512) {
    sqlite3_stmt* stmt;
    const char* sql = "INSERT OR IGNORE INTO safe_files VALUES (?);";
    sqlite3_prepare_v2(safe_db.db, sql, -1, &stmt, nullptr);
    sqlite3_bind_text(stmt, 1, sha512.c_str(), -1, SQLITE_STATIC);
    sqlite3_step(stmt);
    sqlite3_finalize(stmt);
}

/* ========================   Scan History   ========================
*/
bool init_scan_history(ScanHistoryDB& db, const std::string& path) {
    if (sqlite3_open(path.c_str(), &db.db) != SQLITE_OK) return false;
    const char* sql =
        "CREATE TABLE IF NOT EXISTS scan_sessions (id INTEGER PRIMARY
KEY AUTOINCREMENT, sid_string TEXT, type TEXT, timestamp TEXT);"
        "CREATE TABLE IF NOT EXISTS scan_files (session_id INTEGER,
```

```
path TEXT, verdict TEXT, ml REAL, heur REAL);";
    return sqlite3_exec(db.db, sql, nullptr, nullptr, nullptr) ==
SQLITE_OK;
}

int create_scan_session(ScanHistoryDB& db, const std::string&
scan_type, const std::string& sid_string) {
    sqlite3_stmt* stmt;
    const char* sql = "INSERT INTO scan_sessions(sid_string, type,
timestamp) VALUES(?,?,?);";
    sqlite3_prepare_v2(db.db, sql, -1, &stmt, nullptr);
    sqlite3_bind_text(stmt, 1, sid_string.c_str(), -1, SQLITE_STATIC);
    sqlite3_bind_text(stmt, 2, scan_type.c_str(), -1, SQLITE_STATIC);
    std::string ts = now_utc();
    sqlite3_bind_text(stmt, 3, ts.c_str(), -1, SQLITE_STATIC);
    sqlite3_step(stmt);
    sqlite3_finalize(stmt);
    return (int)sqlite3_last_insert_rowid(db.db);
}

void log_file_scan(ScanHistoryDB& db, int session_id, const ScanResult&
r) {
    sqlite3_stmt* stmt;
    const char* sql = "INSERT INTO scan_files VALUES (?,?,?,?,?);";
    sqlite3_prepare_v2(db.db, sql, -1, &stmt, nullptr);
    sqlite3_bind_int(stmt, 1, session_id);
    sqlite3_bind_text(stmt, 2, r.file_path.c_str(), -1, SQLITE_STATIC);
    sqlite3_bind_text(stmt, 3, r.verdict.c_str(), -1, SQLITE_STATIC);
    sqlite3_bind_double(stmt, 4, r.ml_score);
    sqlite3_bind_double(stmt, 5, r.heuristic_score);
    sqlite3_step(stmt);
    sqlite3_finalize(stmt);
}

void log_scan_summary(ScanHistoryDB& db, int session_id, int
total_files, int threats) {
    sqlite3_stmt* stmt;
    const char* sql = "UPDATE scan_sessions SET timestamp = timestamp
|| ' | Scanned: ' || ? || ' | Threats: ' || ? WHERE id = ?;";
    if (sqlite3_prepare_v2(db.db, sql, -1, &stmt, nullptr) ==
SQLITE_OK) {
        sqlite3_bind_int(stmt, 1, total_files);
        sqlite3_bind_int(stmt, 2, threats);
        sqlite3_bind_int(stmt, 3, session_id);
        sqlite3_step(stmt);
        sqlite3_finalize(stmt);
    }
}
```

```
/* ========================   Scan Engine   ========================
*/
ScanResult scan_file(const std::string& path, SafeDB& safe_db, const
MLInference& ml, ScanHistoryDB& history_db, int session_id, const
std::string& sid_string) {
    ScanResult r;
    r.file_path = path;
    std::string ts = now_utc();

    if (!compute_hashes(path, r.sha256, r.sha512)) {
        r.verdict = "error";
        log_event_json(sid_string, ts, path, r.verdict);
        return r;
    }

    if (safe_db_contains(safe_db, r.sha512)) {
        r.verdict = "safe";
        log_event_json(sid_string, ts, path, r.verdict);
        return r;
    }

    r.signature_hit = signature_hit(r.sha256);
    r.ml_score = ml.predict(extract_features(path));
    r.heuristic_score = run_heuristic_scan(path);

    // Competitive scoring: The engine trusts the highest confidence
source
    r.final_score = (std::max)(r.ml_score, r.heuristic_score);

    // --- UPGRADED VERDICT LOGIC (Gray Zone) ---
    // High Priority: Known signatures or score 0.8 - 1.0
    if (r.signature_hit || r.final_score == 0.8) {
        r.is_malicious = true;
        r.verdict = "malicious";
        handle_malicious_file(path, sid_string, r.sha256, r.sha512);
    }
    // Medium Priority: Suspicious threshold
    else if (r.final_score >= 0.65) {
        r.is_malicious = false;
        r.verdict = "suspicious";
        // Suspicious files are logged but not quarantined to avoid FPs
    }
    // Low Priority: Safe
    else {
        r.is_malicious = false;
        r.verdict = "clean";
        safe_db_mark(safe_db, r.sha512);
    }
```

```
    log_file_scan(history_db, session_id, r);
    log_event_json(sid_string, ts, path, r.verdict);
    return r;
}

void scan_recursive(const std::string& root, SafeDB& safe_db, const
MLInference& ml, ScanHistoryDB& history_db, int session_id, const
std::string& sid_string, int& files, int& threats) {
    if (!fs::exists(root)) return;
    for (const auto& e : fs::recursive_directory_iterator(root,
fs::directory_options::skip_permission_denied)) {
        if (!e.is_regular_file()) continue;
        files++;
        ScanResult r = scan_file(e.path().string(), safe_db, ml,
history_db, session_id, sid_string);
        if (r.is_malicious) threats++;

        // --- CPU CONTROL ---
        // Prevents 99% CPU usage by yielding time back to the OS
        std::this_thread::sleep_for(std::chrono::milliseconds(5));
    }
}

/* ========================   Compatibility Overloads
======================== */
int create_scan_session(ScanHistoryDB& db, const std::string&
scan_type) {
    std::string prefix = (scan_type == "full") ? "FS-" : (scan_type ==
"quick") ? "QS-" : "CS-";
    return create_scan_session(db, scan_type, prefix +
std::to_string(1000 + (rand() % 9000)));
}

ScanResult scan_file(const std::string& path, SafeDB& safe_db, const
MLInference& ml, ScanHistoryDB& history_db, int session_id) {
    return scan_file(path, safe_db, ml, history_db, session_id, "UNK-
0000");
}

void scan_recursive(const std::string& root, SafeDB& safe_db, const
MLInference& ml, ScanHistoryDB& history_db, int session_id, int& files,
int& threats) {
    scan_recursive(root, safe_db, ml, history_db, session_id, "UNK-
0000", files, threats);
}

void scan_full(SafeDB& safe_db, const MLInference& ml, ScanHistoryDB&
history_db, const std::string& sid) {
    int files = 0, threats = 0;
```

```
    int session_id = create_scan_session(history_db, "full", sid);
    scan_recursive("C:\\", safe_db, ml, history_db, session_id, sid,
files, threats);
    log_scan_summary(history_db, session_id, files, threats);
}

void scan_quick(SafeDB& safe_db, const MLInference& ml, ScanHistoryDB&
history_db, const std::string& sid) {
    int files = 0, threats = 0;
    int session_id = create_scan_session(history_db, "quick", sid);

    char* userprofile = nullptr;
    size_t len = 0;
    if (_dupenv_s(&userprofile, &len, "USERPROFILE") == 0 &&
userprofile) {
        std::string base(userprofile);
        scan_recursive(base + "\\Downloads", safe_db, ml, history_db,
session_id, sid, files, threats);
        scan_recursive(base + "\\Desktop", safe_db, ml, history_db,
session_id, sid, files, threats);
        scan_recursive(base + "\\Documents", safe_db, ml, history_db,
session_id, sid, files, threats);
        free(userprofile);
    }
    log_scan_summary(history_db, session_id, files, threats);
}
```

VantorBackend.h

```
#pragma once
#include <string>
#include <vector>
#include <mutex>
#include <sqlite3.h>
#include <nlohmann/json.hpp>

using json = nlohmann::json;

/* ========================
   Database Contexts
   ======================== */

struct SafeDB {
    sqlite3* db = nullptr;
};

struct ScanHistoryDB {
    sqlite3* db = nullptr;
};
```

```
/* ========================
   Core Scan Result Struct
   ======================== */

struct ScanResult {
    std::string file_path;
    std::string sha256;
    std::string sha512;

    bool is_malicious = false;
    bool signature_hit = false;

    double ml_score = 0.0;
    double heuristic_score = 0.0;  // Now reflects weighted points
    double final_score = 0.0;

    std::string verdict;           // "clean", "suspicious",
"malicious"
};

class MLInference; // Forward declaration

/* ========================
   Safe DB Interfaces
   ======================== */

bool safe_db_init(SafeDB& safe_db, const std::string& path);
bool safe_db_contains(SafeDB& safe_db, const std::string& sha512);
void safe_db_mark(SafeDB& safe_db, const std::string& sha512);

/* ========================
   Scan History Interfaces
   ======================== */

bool init_scan_history(ScanHistoryDB& db, const std::string& path);

// FIX: Added sid_string to match implementation
int create_scan_session(ScanHistoryDB& db, const std::string&
scan_type, const std::string& sid_string);

void log_file_scan(ScanHistoryDB& db, int session_id, const ScanResult&
result);
void log_scan_summary(ScanHistoryDB& db, int session_id, int
total_files, int threats);

/* ========================
   Scanning Functions
   ======================== */
```

```
   // FIX: Added sid_string to solve C2660/C3861 errors
ScanResult scan_file(
    const std::string& path,
    SafeDB& safe_db,
    const MLInference& ml,
    ScanHistoryDB& history_db,
    int session_id,
    const std::string& sid_string = "UNK-0000"
);

// FIX: Added sid_string (Total 8 arguments to match .cpp
implementation)
void scan_recursive(
    const std::string& root,
    SafeDB& safe_db,
    const MLInference& ml,
    ScanHistoryDB& history_db,
    int session_id,
    const std::string& sid_string,
    int& files_scanned,
    int& threats_found
);

/* ========================
   Central JSONL Logging
   ======================== */

void log_event_json(const std::string& sid, const std::string&
timestamp, const std::string& path, const std::string& verdict);
void log_event_json(const nlohmann::json& obj);

/* ========================
   Compatibility Overloads
   ======================== */
   // These allow older code to still call functions without the SID
string
int create_scan_session(ScanHistoryDB& db, const std::string&
scan_type);
void scan_custom(const std::string& path, SafeDB& safe_db, const
MLInference& ml, ScanHistoryDB& history_db, const std::string& sid);
void scan_recursive(const std::string& root, SafeDB& safe_db, const
MLInference& ml, ScanHistoryDB& history_db, int session_id, int& files,
int& threats);
```

custom.cpp

```
#include "mlinference.h"
#include "VantorBackend.h"
```

```
#include <filesystem>

namespace fs = std::filesystem;

void scan_custom(const std::string& path, SafeDB& safe_db, const
MLInference& ml, ScanHistoryDB& history_db, const std::string& sid) {
    int files_scanned = 0;
    int threats_found = 0;
    if (!fs::exists(path)) return;

    // Use the 3-argument version of create_scan_session
    int session_id = create_scan_session(history_db, "custom", sid);

    // Provide all 8 required arguments to scan_recursive
    scan_recursive(path, safe_db, ml, history_db, session_id, sid,
files_scanned, threats_found);

    log_scan_summary(history_db, session_id, files_scanned,
threats_found);
}
```

defrag.cpp

```
#include <windows.h>
#include <filesystem>
#include <thread>
#include <chrono>
#include <vector>

namespace fs = std::filesystem;
namespace chrono = std::chrono;

// ---------------- CONFIG ----------------
constexpr int LOOP_DELAY_MS = 10 * 60 * 1000; // 10 minutes

// Helper: check if drive is ready
bool drive_ready(const wchar_t drive) {
    wchar_t root[] = { drive, L':', L'\\', 0 };
    UINT type = GetDriveTypeW(root);
    return type == DRIVE_FIXED || type == DRIVE_REMOVABLE;
}

// Defrag a single file (lightweight: read/write to trigger OS caching)
void defrag_file(const fs::path& path) {
    HANDLE h = CreateFileW(
        path.c_str(),
        GENERIC_READ | GENERIC_WRITE,
        FILE_SHARE_READ | FILE_SHARE_WRITE,
```

```
        nullptr,
        OPEN_EXISTING,
        FILE_FLAG_NO_BUFFERING | FILE_FLAG_WRITE_THROUGH,
        nullptr
    );
    if (h != INVALID_HANDLE_VALUE) CloseHandle(h);
}

// Main cleanup loop
void run_defrag_background() {
    while (true) {
        try {
            for (wchar_t drive = L'C'; drive <= L'Z'; ++drive) {
                if (!drive_ready(drive)) continue;

                wchar_t root[] = { drive, L':', L'\\', 0 };
                for (auto& e : fs::recursive_directory_iterator(root,
fs::directory_options::skip_permission_denied)) {
                    try {
                        if (fs::is_regular_file(e.path()))
                            defrag_file(e.path());
                    } catch (...) { continue; }
                }
            }
        } catch (...) { /* resilient to any filesystem errors */ }


std::this_thread::sleep_for(chrono::milliseconds(LOOP_DELAY_MS));
    }
}
```

disk.cpp

```
// disk.cpp
#include <windows.h>
#include <filesystem>
#include <vector>
#include <algorithm>
#include <chrono>
#include <cstdlib>

namespace fs = std::filesystem;
namespace chrono = std::chrono;

// ------------------ PER-DIR CAPS (20% TIGHTER) ------------------
constexpr uint64_t CAP_D3DS_MB = 100;
constexpr uint64_t CAP_TEMP_MB = 200;
constexpr uint64_t CAP_DUMPS_MB = 100;
```

```
constexpr uint64_t CAP_WIN_TEMP_MB = 200;

// ------------------ AGE THRESHOLDS ------------------
constexpr uint64_t AGE_GENERAL_DAYS = 5;
constexpr uint64_t AGE_JUNK_HOURS = 24;

// ------------------ SIZE HEURISTICS ------------------
constexpr uint64_t BIG_SHADER_MB = 5;

struct Stats {
    uint64_t scanned = 0;
    uint64_t deleted = 0;
    uint64_t freed_bytes = 0;
    uint64_t skipped_locked = 0;
    uint64_t skipped_perm = 0;
};

bool file_locked(const fs::path& p) {
    HANDLE h = CreateFileW(
        p.c_str(),
        GENERIC_READ,
        FILE_SHARE_READ | FILE_SHARE_WRITE | FILE_SHARE_DELETE,
        nullptr,
        OPEN_EXISTING,
        0,
        nullptr
    );
    if (h == INVALID_HANDLE_VALUE) return true;
    CloseHandle(h);
    return false;
}

bool multi_hardlink(const fs::path& p) {
    HANDLE h = CreateFileW(
        p.c_str(),
        0,
        FILE_SHARE_READ | FILE_SHARE_WRITE | FILE_SHARE_DELETE,
        nullptr,
        OPEN_EXISTING,
        FILE_FLAG_BACKUP_SEMANTICS,
        nullptr
    );
    if (h == INVALID_HANDLE_VALUE) return true;

    BY_HANDLE_FILE_INFORMATION info{};
    GetFileInformationByHandle(h, &info);
    CloseHandle(h);
    return info.nNumberOfLinks > 1;
}
```

```
bool is_junk_ext(const fs::path& p) {
    auto ext = p.extension().wstring();
    return ext == L".tmp" || ext == L".log" || ext == L".dmp" || ext ==
L".bak";
}

uint64_t file_age_hours(const fs::file_time_type& ft) {
    auto now = fs::file_time_type::clock::now();
    return chrono::duration_cast<chrono::hours>(now - ft).count();
}

void cap_cache(const fs::path& dir, uint64_t cap_mb, Stats& st) {
    if (!fs::exists(dir)) return;

    struct Entry {
        fs::path p;
        uint64_t size;
        fs::file_time_type time;
    };

    std::vector<Entry> files;
    uint64_t total = 0;

    try {
        for (const auto& e : fs::directory_iterator(
            dir, fs::directory_options::skip_permission_denied)) {

            if (!e.is_regular_file()) continue;
            st.scanned++;

            const fs::path& p = e.path();

            if (file_locked(p)) { st.skipped_locked++; continue; }
            if (multi_hardlink(p)) continue;

            uint64_t size = 0;
            try { size = fs::file_size(p); }
            catch (...) { st.skipped_perm++; continue; }

            auto ftime = e.last_write_time();
            uint64_t age_h = file_age_hours(ftime);

            bool junk = is_junk_ext(p) && age_h >= AGE_JUNK_HOURS;
            bool old = age_h >= (AGE_GENERAL_DAYS * 24);
            bool big_shader =
                (size >= (BIG_SHADER_MB * 1024 * 1024)) &&
                p.extension() == L".bin";
```

```
            if (!(junk || old || big_shader))
                continue;

            total += size;
            files.push_back({ p, size, ftime });
        }
    }
    catch (...) {
        return;
    }

    std::sort(files.begin(), files.end(),
        [](const Entry& a, const Entry& b) {
            return a.time < b.time;
        });

    uint64_t limit = cap_mb * 1024 * 1024;

    for (const auto& f : files) {
        if (total <= limit) break;

        try {
            fs::remove(f.p);
            total -= f.size;
            st.deleted++;
            st.freed_bytes += f.size;
        }
        catch (...) {}
    }
}

// ============================================================
// PUBLIC ENTRY POINT
// ============================================================
void run_disk_cleanup() {
    Stats st;

    wchar_t* local = nullptr;
    size_t len = 0;
    if (_wdupenv_s(&local, &len, L"LOCALAPPDATA") != 0 || !local)
        return;

    cap_cache(fs::path(local) / L"D3DSCache", CAP_D3DS_MB, st);
    cap_cache(fs::path(local) / L"Temp", CAP_TEMP_MB, st);
    cap_cache(fs::path(local) / L"CrashDumps", CAP_DUMPS_MB, st);
    cap_cache(L"C:\\Windows\\Temp", CAP_WIN_TEMP_MB, st);

    free(local);
```

```
}
```

drive.cpp

```
#define _CRT_SECURE_NO_WARNINGS
#include "VantorBackend.h"
#include "quarantine.h"
#include "mlinference.h"
#include "VantorBackend.h"
#include "logging.h"
#include <Windows.h>
#include <filesystem>
#include <vector>
#include <string>

namespace fs = std::filesystem;

void scan_drive(const std::string& drive_letter, SafeDB& safe,
MLInference& ml, ScanHistoryDB& history_db, const std::string& sid) {
    if (!fs::exists(drive_letter)) return;
    int files_scanned = 0;
    int threats_found = 0;

    int session_id = create_scan_session(history_db, "drive", sid);

    for (auto& entry : fs::recursive_directory_iterator(drive_letter,
fs::directory_options::skip_permission_denied)) {
        if (entry.is_regular_file()) {
            // Pass the sid to scan_file
            ScanResult res = scan_file(entry.path().string(), safe, ml,
history_db, session_id, sid);
            files_scanned++;
            if (res.is_malicious) threats_found++;
        }
    }
    log_scan_summary(history_db, session_id, files_scanned,
threats_found);
}
```

feature_extractor.cpp

```
#define NOMINMAX
#include "feature_extractor.h"
#include <windows.h>
#include <fstream>
#include <vector>
#include <cmath>
#include <algorithm>
```

```
/**
 * Calculates normalized entropy (0.0 to 1.0) to identify potentially
 * packed or encrypted data segments [cite: 590-594].
 */
static double calculate_entropy(const uint8_t* data, size_t size) {
    if (size == 0) return 0.0;
    double freq[256] = {};
    for (size_t i = 0; i < size; i++)
        freq[data[i]]++;
    double ent = 0.0;
    for (double f : freq) {
        if (f == 0) continue;
        double p = f / size;
        ent -= p * std::log2(p);
    }
    return ent / 8.0;
}

/**
 * Helper to read raw binary data into a buffer [cite: 594-595].
 */
static bool read_file(const std::string& path, std::vector<uint8_t>&
out) {
    std::ifstream f(path, std::ios::binary);
    if (!f) return false;
    out.assign(std::istreambuf_iterator<char>(f), {});
    return true;
}

/**
 * Deep PE parsing to extract structural and behavioral traits [cite:
596-597].
 * Upgraded to identify 'Suspicious Sections' (e.g., Writable +
Executable).
 */
template<typename NT_HEADERS>
static void parse_pe_internal(const std::vector<uint8_t>& buf,
std::vector<float>& feats, const NT_HEADERS* nt) {
    const auto& fh = nt->FileHeader;
    const auto& oh = nt->OptionalHeader;

    // Structural features: Section count, Characteristics, and Memory
layout
    feats.push_back((float)fh.NumberOfSections / 10.0f);
    feats.push_back((float)fh.Characteristics / 65535.0f);
    feats.push_back((float)oh.SizeOfImage / (1024.0f * 1024.0f));
    feats.push_back((float)oh.AddressOfEntryPoint / (1024.0f *
1024.0f));
```

```
    const IMAGE_SECTION_HEADER* sec = IMAGE_FIRST_SECTION(nt);
    double max_ent = 0.0;
    double mean_ent = 0.0;
    int suspicious_sections = 0;

    for (int i = 0; i < fh.NumberOfSections; i++) {
        if (sec[i].PointerToRawData + sec[i].SizeOfRawData >
buf.size())
            continue;

        // UPGRADE: Identify sections that are both Writable and
Executable (W^X violation)
        // This is a hallmark of unpacked malware or shellcode stubs .
        if ((sec[i].Characteristics & IMAGE_SCN_MEM_WRITE) &&
            (sec[i].Characteristics & IMAGE_SCN_MEM_EXECUTE)) {
            suspicious_sections++;
        }

        double e = calculate_entropy(&buf[sec[i].PointerToRawData],
sec[i].SizeOfRawData);
        max_ent = (std::max)(max_ent, e);
        mean_ent += e;
    }

    if (fh.NumberOfSections > 0)
        mean_ent /= fh.NumberOfSections;

    feats.push_back((float)mean_ent);
    feats.push_back((float)max_ent);

    // UPGRADE: New features for the Weighted Point System
    feats.push_back((float)suspicious_sections / 5.0f);
    feats.push_back((float)oh.CheckSum == 0 ? 1.0f : 0.0f); // Missing
checksums are common in custom malware
}

std::vector<float> extract_features(const std::string& path) {
    std::vector<float> features;
    features.reserve(8);
    std::vector<uint8_t> buf;

    if (!read_file(path, buf) || buf.size() < sizeof(IMAGE_DOS_HEADER))
{
        return std::vector<float>(8, 0.0f);
    }

    auto* dos = (IMAGE_DOS_HEADER*)buf.data();
    if (dos->e_magic != IMAGE_DOS_SIGNATURE || dos->e_lfanew <= 0 ||
```

```
        dos->e_lfanew + sizeof(DWORD) > buf.size()) {
        return std::vector<float>(8, 0.0f);
    }

    DWORD nt_sig = *(DWORD*)&buf[dos->e_lfanew];
    if (nt_sig != IMAGE_NT_SIGNATURE) {
        return std::vector<float>(8, 0.0f);
    }

    WORD magic = *(WORD*)(&buf[dos->e_lfanew + sizeof(DWORD) +
sizeof(IMAGE_FILE_HEADER)]);

    if (magic == IMAGE_NT_OPTIONAL_HDR32_MAGIC) {
        parse_pe_internal(buf, features, (IMAGE_NT_HEADERS32*)&buf[dos-
>e_lfanew]);
    }
    else if (magic == IMAGE_NT_OPTIONAL_HDR64_MAGIC) {
        parse_pe_internal(buf, features, (IMAGE_NT_HEADERS64*)&buf[dos-
>e_lfanew]);
    }
    else {
        features.assign(8, 0.0f);
    }

    // Ensure we always return at least the minimum expected vector
size
    if (features.size() < 8)
        features.resize(8, 0.0f);

    return features;
}
```

feature_extractor.h

```
#pragma once
#include <vector>
#include <string>

std::vector<float> extract_features(const std::string& path);
```

heuristic.cpp

```
#include "heuristic.h"
#include "VantorBackend.h"
#include <windows.h>
#include <wintrust.h>
#include <softpub.h>
#include <filesystem>
```

```
#include <string>
#include <algorithm>

#pragma comment(lib, "wintrust.lib")

/**
 * Checks if a file has a valid digital signature.
 * Trusted signatures are the strongest defense against False
Positives.
 */
static bool is_signed_and_valid(const std::wstring& path) {
    WINTRUST_FILE_INFO fileInfo{};
    fileInfo.cbStruct = sizeof(fileInfo);
    fileInfo.pcwszFilePath = path.c_str();

    WINTRUST_DATA trustData{};
    trustData.cbStruct = sizeof(trustData);
    trustData.dwUIChoice = WTD_UI_NONE;
    trustData.fdwRevocationChecks = WTD_REVOKE_NONE;
    trustData.dwUnionChoice = WTD_CHOICE_FILE;
    trustData.pFile = &fileInfo;
    trustData.dwStateAction = WTD_STATEACTION_VERIFY;
    trustData.dwProvFlags = WTD_CACHE_ONLY_URL_RETRIEVAL;

    GUID policy = WINTRUST_ACTION_GENERIC_VERIFY_V2;
    LONG status = WinVerifyTrust(nullptr, &policy, &trustData);

    // Cleanup
    trustData.dwStateAction = WTD_STATEACTION_CLOSE;
    WinVerifyTrust(nullptr, &policy, &trustData);

    return status == ERROR_SUCCESS;
}

/**
 * UPGRADED: Weighted Heuristic Engine
 * Instead of simple addition, we use a point system normalized to 0.0
- 1.0.
 */
double run_heuristic_scan(const std::string& path) {
    int total_points = 0;
    std::error_code ec;
    auto file_size = std::filesystem::file_size(path, ec);

    // 1. Digital Signature Logic (The primary FP filter)
    std::wstring wpath(path.begin(), path.end());
    if (is_signed_and_valid(wpath)) {
        // Massive point reduction for signed files.
        // This makes it nearly impossible for a signed Microsoft app
```

```
to be flagged.
        total_points -= 60;
    }
    else {
        // Penalty for unsigned binaries.
        total_points += 30;
    }

    // 2. Structural/Size Heuristics
    if (!ec) {
        if (file_size < 10 * 1024) total_points += 20;       // Tiny
stubs/droppers
        if (file_size > 150 * 1024 * 1024) total_points += 5; //
Unusually bloated
    }

    // 3. Contextual Heuristics (Where is the file?)
    // Malware often executes from Temp or AppData to bypass
permissions.
    std::string path_lower = path;
    std::transform(path_lower.begin(), path_lower.end(),
path_lower.begin(), ::tolower);

    if (path_lower.find("\\appdata\\") != std::string::npos ||
        path_lower.find("\\temp\\") != std::string::npos) {
        total_points += 25;
    }

    // 4. Double Extension Check (Classic trick: invoice.pdf.exe)
    if (path_lower.find(".pdf.exe") != std::string::npos ||
        path_lower.find(".txt.exe") != std::string::npos ||
        path_lower.find(".jpg.exe") != std::string::npos) {
        total_points += 50;
    }

    // 5. Extension weighting
    if (path_lower.ends_with(".exe") || path_lower.ends_with(".dll") ||
path_lower.ends_with(".sys")) {
        total_points += 10;
    }

    // Normalize: Convert points to a 0.0 - 1.0 scale
    // 0 points = 0.0, 100+ points = 1.0
    double final_score = static_cast<double>(total_points) / 100.0;

    // Clamp values
    if (final_score < 0.0) final_score = 0.0;
    if (final_score > 1.0) final_score = 1.0;
```

```
    return final_score;
}
```

heuristic.h

```
#pragma once
#include <string>

double run_heuristic_scan(const std::string& path);
```

logging.h

```
#pragma once
#include "VantorBackend.h"
#include "nlohmann/json.hpp"
#include <fstream>
#include <mutex>

using json = nlohmann::json;

static const std::string LOG_FILE = "logs.json";
static std::mutex log_mutex;

// Thread-safe append to logs.#pragma once

#include "VantorBackend.h"

/*
  Thin compatibility wrapper.
  No mutex.
  No file IO.
  No parsing.
*/

inline void log_scan_result(const ScanResult& r) {
    log_event_json({
        {"type", "file_scan_compat"},
        {"path", r.file_path},
        {"sha256", r.sha256},
        {"sha512", r.sha512},
        {"ml", r.ml_score},
        {"heur", r.heuristic_score},
        {"verdict", r.verdict}
        });
}
```

main.cpp

```
#include "VantorBackend.h"
#include "mlinference.h"
#include <iostream>
#include <vector>
#include <string>
#include <ctime>
#include <windows.h> // Required for Sleep

// ================== AV SCAN PROTOTYPES ==================
void scan_full(SafeDB&, const MLInference&, ScanHistoryDB&, const
std::string&);
void scan_quick(SafeDB&, const MLInference&, ScanHistoryDB&, const
std::string&);
void scan_custom(const std::string&, SafeDB&, const MLInference&,
ScanHistoryDB&, const std::string&);
void run_rtp_engine(SafeDB& safe, const MLInference& ml, ScanHistoryDB&
history);

// ================== SOT MODULE ENTRY POINTS ==================
void run_disk_cleanup();           // from disk.cpp
void run_startup_optimization();   // from start_man.cpp

int main(int argc, char* argv[]) {
    // 1. Fast-path: Tools that DON'T require Databases or ML
    if (argc >= 2) {
        std::string cmd = argv[1];
        if (cmd == "clean") {
            std::cout << "[SOT] Disk cleanup invoked\n";
            run_disk_cleanup();
            return 0;
        }
        if (cmd == "mon") {
            std::cout << "[SOT] Startup manager invoked\n";
            run_startup_optimization();
            return 0;
        }
    }

    // 2. Initialize Core Engines (Required for Scans and RTP Service)
    SafeDB safe_db;
    ScanHistoryDB history_db;
    MLInference ml;

    srand((unsigned int)time(NULL));

    if (!safe_db_init(safe_db, "safe.db")) {
        std::cerr << "[FATAL] Failed to init SafeDB\n";
        return 1;
```

```
    }

    if (!init_scan_history(history_db, "scan_history.db")) {
        std::cerr << "[FATAL] Failed to init scan history DB\n";
        return 1;
    }

    std::vector<std::string> models = {
        "ember.json"
    };

    if (!ml.load_models(models)) {
        std::cerr << "[FATAL] Failed to load ML models\n";
        return 1;
    }

    // 3. Handle Service Mode (Must happen after ML/DB init)
    if (argc >= 2 && std::string(argv[1]) == "service") {
        log_event_json({ {"type", "rtp_start"}, {"status", "active"}
});

        run_rtp_engine(safe_db, ml, history_db);

        // Keep the process alive
        while (true) {
            Sleep(1000);
        }
        return 0;
    }

    // 4. Handle Standard On-Demand Scans
    if (argc < 2) {
        std::cout << "Vantor Security Suite\n";
        std::cout << "Usage:\n";
        std::cout << "  vantor full           (Full System Scan)\n";
        std::cout << "  vantor quick          (Quick Scan)\n";
        std::cout << "  vantor custom <path>  (Scan specific
folder)\n";
        std::cout << "  vantor service        (Run Real-Time
Protection)\n";
        std::cout << "  vantor clean          (Disk Cleanup)\n";
        std::cout << "  vantor mon            (Startup Optimizer)\n";
        return 0;
    }

    log_event_json({
        {"type", "startup"},
        {"ml_models", (int)models.size()}
        });
```

```
    std::string mode = argv[1];
    std::string prefix = (mode == "full") ? "FS-" : (mode == "quick") ?
"QS-" : "CS-";
    std::string sid = prefix + std::to_string(1000 + (rand() % 9000));

    if (mode == "full") {
        scan_full(safe_db, ml, history_db, sid);
    }
    else if (mode == "quick") {
        scan_quick(safe_db, ml, history_db, sid);
    }
    else if (mode == "custom" && argc >= 3) {
        scan_custom(argv[2], safe_db, ml, history_db, sid);
    }
    else {
        std::cerr << "[ERROR] Invalid arguments or missing path for
custom scan\n";
    }

    log_event_json({ {"type", "shutdown"} });
    return 0;
}
```

memory.cpp

```
#include "VantorBackend.h"
#include <string>

void log_memory_event(
    const std::string& type,
    const std::string& message
) {
    log_event_json({
        {"type", type},
        {"message", message}
        });
}
```

mlinference.cpp

```
#include "mlinference.h"
#include "VantorBackend.h"
#include <xgboost/c_api.h>
#include <algorithm>

MLInference::~MLInference() {
    for (void* booster : boosters) {
```

```
        XGBoosterFree((BoosterHandle)booster);
    }
}

bool MLInference::load_models(const std::vector<std::string>&
model_paths) {
    boosters.clear();
    for (const auto& path : model_paths) {
        BoosterHandle booster = nullptr;
        if (XGBoosterCreate(nullptr, 0, &booster) != 0) {
            log_event_json({ {"type","ml_error"},{"msg","BoosterCreate
failed"} });
            return false;
        }
        if (XGBoosterLoadModel(booster, path.c_str()) != 0) {
            log_event_json({ {"type","ml_error"},{"msg","LoadModel
failed"},{"path",path} });
            XGBoosterFree(booster);
            return false;
        }
        XGBoosterSetParam(booster, "nthread", "2");
        boosters.push_back((void*)booster);
    }
    log_event_json({ {"type","ml_init"},
{"models_loaded",(int)boosters.size()} });
    return !boosters.empty();
}

double MLInference::predict(const std::vector<float>& features) const {
    if (boosters.empty() || features.empty())
        return 0.0;

    double sum = 0.0;
    for (size_t i = 0; i < boosters.size(); ++i) {
        void* booster = boosters[i];
        DMatrixHandle mat;
        // Corrected bst_ulong cast
        XGDMatrixCreateFromMat(features.data(), 1,
(bst_ulong)features.size(), -1.0f, &mat);

        bst_ulong out_len = 0;
        const float* out = nullptr;
        // Corrected XGBoosterPredict signature with 6 arguments
        XGBoosterPredict((BoosterHandle)booster, mat, 0, 0, 0,
&out_len, &out);

        if (out_len > 0) {
            float score = std::clamp(out[0], 0.0f, 1.0f);
            sum += score;
```

```
            // NEW LOGGING LOGIC
            // Logs individual model prediction to logs.jsonl
            log_event_json({
                {"type", "ml_score_prediction"},
                {"model_index", (int)i},
                {"score", (double)score}
                });
        }

        XGDMatrixFree(mat);
    }

    double final_avg = sum / boosters.size();

    return final_avg;
}
```

mlinference.h

```
#pragma once
#include <vector>
#include <string>

/*
  Thin ML inference wrapper around XGBoost JSON models.
  - Supports multiple models (ensemble)
  - Deterministic
  - Safe failure behavior
*/

class MLInference {
public:
    bool load_models(const std::vector<std::string>& model_paths);
    double predict(const std::vector<float>& features) const;
    bool is_ready() const { return !boosters.empty(); }

    ~MLInference();

private:
    std::vector<void*> boosters; // XGBoost booster handles
};
```

quarantine.cpp

```
#include "quarantine.h"
#include "VantorBackend.h"
#include <windows.h>
```

```
#include <wincrypt.h>
#include <filesystem>
#include <fstream>
#include <vector>
#include <chrono>
#include <iomanip>
#include <sstream>
#include <openssl/evp.h>
#include <openssl/rand.h>
#include <sqlite3.h>

#pragma comment(lib, "crypt32.lib")
namespace fs = std::filesystem;

static const size_t KEY_SIZE = 32;
static const size_t IV_SIZE = 12;
static const size_t TAG_SIZE = 16;

/* ========================   Helpers   ======================== */

static std::string get_current_timestamp() {
    auto now = std::chrono::system_clock::now();
    std::time_t t = std::chrono::system_clock::to_time_t(now);
    std::tm tm;
    gmtime_s(&tm, &t);
    std::ostringstream oss;
    oss << std::put_time(&tm, "%Y-%m-%dT%H:%M:%SZ");
    return oss.str();
}

std::string get_quarantine_dir() {
    fs::path dir = fs::current_path() / "quarantine";
    fs::create_directories(dir);
    return dir.string();
}

/* ========================   Database Logic
======================== */

static void init_quarantine_db() {
    sqlite3* db;
    std::string db_path = (fs::path(get_quarantine_dir()) /
"quarantine.db").string();
    if (sqlite3_open(db_path.c_str(), &db) == SQLITE_OK) {
        const char* sql = "CREATE TABLE IF NOT EXISTS quarantined_files
("
            "id INTEGER PRIMARY KEY AUTOINCREMENT, "
            "timestamp TEXT, "
            "scan_id TEXT, "
```

```
            "original_path TEXT, "
            "sha256 TEXT, "
            "sha512 TEXT);";
        sqlite3_exec(db, sql, nullptr, nullptr, nullptr);
    }
    sqlite3_close(db);
}

static void log_to_quarantine_db(const std::string& sid, const
std::string& path, const std::string& s256, const std::string& s512) {
    sqlite3* db;
    std::string db_path = (fs::path(get_quarantine_dir()) /
"quarantine.db").string();
    if (sqlite3_open(db_path.c_str(), &db) == SQLITE_OK) {
        const char* sql = "INSERT INTO quarantined_files (timestamp,
scan_id, original_path, sha256, sha512) VALUES (?, ?, ?, ?, ?);";
        sqlite3_stmt* stmt;
        if (sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr) ==
SQLITE_OK) {
            std::string ts = get_current_timestamp();
            sqlite3_bind_text(stmt, 1, ts.c_str(), -1,
SQLITE_TRANSIENT);
            sqlite3_bind_text(stmt, 2, sid.c_str(), -1,
SQLITE_TRANSIENT);
            sqlite3_bind_text(stmt, 3, path.c_str(), -1,
SQLITE_TRANSIENT);
            sqlite3_bind_text(stmt, 4, s256.c_str(), -1,
SQLITE_TRANSIENT);
            sqlite3_bind_text(stmt, 5, s512.c_str(), -1,
SQLITE_TRANSIENT);
            sqlite3_step(stmt);
        }
        sqlite3_finalize(stmt);
    }
    sqlite3_close(db);
}

static void add_to_signatures_db(const std::string& sha256) {
    sqlite3* db;
    // Open signatures.db in the current working directory
    if (sqlite3_open("signatures.db", &db) == SQLITE_OK) {
        sqlite3_exec(db, "CREATE TABLE IF NOT EXISTS signatures (hash
TEXT UNIQUE);", nullptr, nullptr, nullptr);
        const char* sql = "INSERT OR IGNORE INTO signatures (hash)
VALUES (?);";
        sqlite3_stmt* stmt;
        if (sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr) ==
SQLITE_OK) {
            sqlite3_bind_text(stmt, 1, sha256.c_str(), -1,
```

```
SQLITE_TRANSIENT);
            sqlite3_step(stmt);
        }
        sqlite3_finalize(stmt);
    }
    sqlite3_close(db);
}

/* ========================   Encryption Logic
======================== */

static bool load_or_create_key(std::vector<uint8_t>& key) {
    fs::path key_path = fs::path(get_quarantine_dir()) / "master.key";

    if (fs::exists(key_path)) {
        std::ifstream in(key_path, std::ios::binary);
        std::vector<uint8_t> enc((std::istreambuf_iterator<char>(in)),
{});
        DATA_BLOB in_blob{ static_cast<DWORD>(enc.size()), enc.data()
};
        DATA_BLOB out_blob{};

        if (!CryptUnprotectData(&in_blob, nullptr, nullptr, nullptr,
nullptr, 0, &out_blob))
            return false;

        key.assign(out_blob.pbData, out_blob.pbData + out_blob.cbData);
        LocalFree(out_blob.pbData);
        return true;
    }

    key.resize(KEY_SIZE);
    RAND_bytes(key.data(), (int)KEY_SIZE);
    DATA_BLOB in_blob{ (DWORD)KEY_SIZE, key.data() };
    DATA_BLOB out_blob{};

    if (!CryptProtectData(&in_blob, L"VantorKey", nullptr, nullptr,
nullptr, 0, &out_blob))
        return false;

    std::ofstream out(key_path, std::ios::binary);
    out.write((char*)out_blob.pbData, out_blob.cbData);
    LocalFree(out_blob.pbData);
    return true;
}

static bool encrypt_file(const std::string& src, const std::string&
dst, const std::vector<uint8_t>& key) {
    std::ifstream in(src, std::ios::binary);
```

```
    if (!in) return false;

    std::vector<uint8_t> data((std::istreambuf_iterator<char>(in)),
{});
    std::vector<uint8_t> iv(IV_SIZE);
    RAND_bytes(iv.data(), (int)IV_SIZE);

    EVP_CIPHER_CTX* ctx = EVP_CIPHER_CTX_new();
    std::vector<uint8_t> out_buf(data.size() + 32);
    int len = 0, outlen = 0;

    EVP_EncryptInit_ex(ctx, EVP_aes_256_gcm(), nullptr, nullptr,
nullptr);
    EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_SET_IVLEN, (int)IV_SIZE,
nullptr);
    EVP_EncryptInit_ex(ctx, nullptr, nullptr, key.data(), iv.data());

    EVP_EncryptUpdate(ctx, out_buf.data(), &len, data.data(),
(int)data.size());
    outlen = len;

    EVP_EncryptFinal_ex(ctx, out_buf.data() + outlen, &len);
    outlen += len;

    std::vector<uint8_t> tag(TAG_SIZE);
    EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_GET_TAG, (int)TAG_SIZE,
tag.data());
    EVP_CIPHER_CTX_free(ctx);

    std::ofstream ofs(dst, std::ios::binary);
    ofs.write((char*)iv.data(), iv.size());
    ofs.write((char*)out_buf.data(), outlen);
    ofs.write((char*)tag.data(), tag.size());
    return true;
}

/* ========================   Public API   ========================
*/

bool handle_malicious_file(const std::string& path, const std::string&
sid, const std::string& sha256, const std::string& sha512) {
    init_quarantine_db();

    std::vector<uint8_t> key;
    if (!load_or_create_key(key)) return false;

    fs::path src(path);
    fs::path dst = fs::path(get_quarantine_dir()) /
(src.filename().string() + ".enc");
```

```
    if (!encrypt_file(src.string(), dst.string(), key)) return false;

    // Record metadata and blacklist hash
    log_to_quarantine_db(sid, path, sha256, sha512);
    add_to_signatures_db(sha256);

    fs::remove(src);
    log_event_json(sid, get_current_timestamp(), path, "quarantined");

    return true;
}

bool decrypt_quarantined_file(const std::string& enc_path, const
std::string& output_path) {
    std::vector<uint8_t> key;
    if (!load_or_create_key(key)) return false;

    std::ifstream in(enc_path, std::ios::binary);
    if (!in) return false;

    std::vector<uint8_t> iv(IV_SIZE);
    in.read((char*)iv.data(), IV_SIZE);

    std::vector<uint8_t> enc_data((std::istreambuf_iterator<char>(in)),
{});
    if (enc_data.size() < TAG_SIZE) return false;

    std::vector<uint8_t> tag(enc_data.end() - TAG_SIZE,
enc_data.end());
    enc_data.resize(enc_data.size() - TAG_SIZE);

    EVP_CIPHER_CTX* ctx = EVP_CIPHER_CTX_new();
    std::vector<uint8_t> out_buf(enc_data.size());
    int len = 0, outlen = 0;

    EVP_DecryptInit_ex(ctx, EVP_aes_256_gcm(), nullptr, nullptr,
nullptr);
    EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_SET_IVLEN, (int)IV_SIZE,
nullptr);
    EVP_DecryptInit_ex(ctx, nullptr, nullptr, key.data(), iv.data());

    EVP_DecryptUpdate(ctx, out_buf.data(), &len, enc_data.data(),
(int)enc_data.size());
    outlen = len;

    EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_SET_TAG, (int)TAG_SIZE,
tag.data());
    if (EVP_DecryptFinal_ex(ctx, out_buf.data() + outlen, &len) <= 0) {
```

```
        EVP_CIPHER_CTX_free(ctx);
        return false;
    }

    EVP_CIPHER_CTX_free(ctx);
    std::ofstream ofs(output_path, std::ios::binary);
    ofs.write((char*)out_buf.data(), outlen + len);
    return true;
}
```

quarantine.h

```
#pragma once
#include <string>

bool handle_malicious_file(
    const std::string& path,
    const std::string& sid,
    const std::string& sha256,
    const std::string& sha512
);

bool decrypt_quarantined_file(
    const std::string& enc_path,
    const std::string& output_path
);

std::string get_quarantine_dir();
```

ram.cpp

```
#include <windows.h>
#include <psapi.h>
#include <thread>
#include <chrono>

namespace chrono = std::chrono;

// ---------------- CONFIG ----------------
constexpr int LOOP_DELAY_MS = 5 * 60 * 1000; // 5 minutes
constexpr SIZE_T MIN_FREE_RAM_MB = 200;      // target free RAM

void run_ram_boost_background() {
    while (true) {
        try {
            // Get system memory info
            MEMORYSTATUSEX memInfo{};
            memInfo.dwLength = sizeof(memInfo);
            if (GlobalMemoryStatusEx(&memInfo)) {
```

```
                SIZE_T availMB = memInfo.ullAvailPhys / (1024 * 1024);
                if (availMB < MIN_FREE_RAM_MB) {
                    // Enumerate processes and softly reduce memory
pressure
                    DWORD processes[1024], needed;
                    if (EnumProcesses(processes, sizeof(processes),
&needed)) {
                        int count = needed / sizeof(DWORD);
                        for (int i = 0; i < count; ++i) {
                            HANDLE h = OpenProcess(PROCESS_SET_QUOTA |
PROCESS_TERMINATE | PROCESS_QUERY_INFORMATION, FALSE, processes[i]);
                            if (h) {
                                // Ask OS to trim working set (soft RAM
boost)
                                SetProcessWorkingSetSize(h, (SIZE_T)-1,
(SIZE_T)-1);
                                CloseHandle(h);
                            }
                        }
                    }
                }
            }
        } catch (...) { /* resilient to all errors */ }


std::this_thread::sleep_for(chrono::milliseconds(LOOP_DELAY_MS));
    }
}
```

rtp.py

```
import subprocess
import os

# Get the current folder of this Python script
current_dir = os.path.dirname(os.path.abspath(__file__))

# Use the dynamic path to find the exe in the same folder
exe_path = os.path.join(current_dir, "VantorBackend.exe")

try:
    # FIX: Removed the space before "service"
    subprocess.Popen(
        [exe_path, "service"],
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
        stdin=subprocess.DEVNULL,
        creationflags=subprocess.CREATE_NO_WINDOW
```

```
    )
    print("RTP Service launched successfully in the background.")
except FileNotFoundError:
    print(f"Error: Could not find {exe_path}")
```

scan_full.cpp

```
#include "VantorBackend.h"
#include "mlinference.h"
#include <filesystem>

namespace fs = std::filesystem;

void scan_full(
    SafeDB& safe_db,
    const MLInference& ml,
    ScanHistoryDB& history_db
) {
    int files_scanned = 0;
    int threats_found = 0;

    int session_id = create_scan_session(history_db, "full");

    for (const auto& drive : fs::directory_iterator("C:\\")) {
        std::error_code ec;
        if (!drive.is_directory(ec))
            continue;

        scan_recursive(
            drive.path().string(),
            safe_db,
            ml,
            history_db,
            session_id,
            files_scanned,
            threats_found
        );
    }

    log_scan_summary(
        history_db,
        session_id,
        files_scanned,
        threats_found
    );
}
```

scan_quick.cpp

```
#include "VantorBackend.h"
#include "mlinference.h"
#include <windows.h>
#include <filesystem>

namespace fs = std::filesystem;

static void scan_if_exists(
    const std::string& path,
    SafeDB& safe_db,
    const MLInference& ml,
    ScanHistoryDB& history_db,
    int session_id,
    int& files,
    int& threats
) {
    if (!fs::exists(path)) return;

    scan_recursive(
        path,
        safe_db,
        ml,
        history_db,
        session_id,
        files,
        threats
    );
}

void scan_quick(
    SafeDB& safe_db,
    const MLInference& ml,
    ScanHistoryDB& history_db
) {
    int files_scanned = 0;
    int threats_found = 0;

    int session_id = create_scan_session(history_db, "quick");

    char* userprofile = nullptr;
    size_t len = 0;

    if (_dupenv_s(&userprofile, &len, "USERPROFILE") == 0 &&
userprofile) {
        std::string base(userprofile);

        scan_if_exists(base + "\\Downloads", safe_db, ml, history_db,
session_id, files_scanned, threats_found);
```

```
        scan_if_exists(base + "\\Desktop", safe_db, ml, history_db,
session_id, files_scanned, threats_found);
        scan_if_exists(base + "\\Documents", safe_db, ml, history_db,
session_id, files_scanned, threats_found);

        free(userprofile);
    }

    log_scan_summary(
        history_db,
        session_id,
        files_scanned,
        threats_found
    );
}
```

signatures.cpp

```
#include "signatures.h"

// Implementation to resolve "Function definition not found"
bool signature_hit(const std::string& sha256) {
    // Placeholder logic for database lookups
    // You can extend this to check against a known-malware hash
database
    return false;
}
```

signatures.h

```
#pragma once
#include <string>

bool signature_hit(const std::string& sha256);
```

start_man.cpp

```
// start_man.cpp
#include <windows.h>
#include <string>

struct Stats {
    int scanned = 0;
    int delayed = 0;
};
```

```
bool heavy_app(const std::wstring& cmd) {
    return cmd.find(L"update") != std::wstring::npos ||
        cmd.find(L"telemetry") != std::wstring::npos ||
        cmd.find(L"launcher") != std::wstring::npos ||
        cmd.find(L"helper") != std::wstring::npos;
}

void process_run_key(HKEY root, Stats& st) {
    HKEY h;
    if (RegOpenKeyExW(
        root,
        L"Software\\Microsoft\\Windows\\CurrentVersion\\Run",
        0,
        KEY_ALL_ACCESS,
        &h
    ) != ERROR_SUCCESS)
        return;

    wchar_t name[256];
    wchar_t data[1024];
    DWORD nameSize, dataSize, type;

    while (true) {
        nameSize = 256;
        dataSize = sizeof(data);

        LONG res = RegEnumValueW(
            h,
            0,
            name,
            &nameSize,
            nullptr,
            &type,
            (LPBYTE)data,
            &dataSize
        );

        if (res != ERROR_SUCCESS)
            break;

        st.scanned++;

        std::wstring cmd(data);

        if (heavy_app(cmd)) {
            std::wstring delayedName = std::wstring(name) +
L"_Delayed";

            RegDeleteValueW(h, name);
```

```
            RegSetValueExW(
                h,
                delayedName.c_str(),
                0,
                REG_SZ,
                reinterpret_cast<const BYTE*>(cmd.c_str()),
                static_cast<DWORD>((cmd.size() + 1) * sizeof(wchar_t))
            );

            st.delayed++;
        }
    }

    RegCloseKey(h);
}

// ============================================================
// PUBLIC ENTRY POINT
// ============================================================
void run_startup_optimization() {
    Stats st;

    process_run_key(HKEY_CURRENT_USER, st);
    process_run_key(HKEY_LOCAL_MACHINE, st);
}
```

startup_monitor.py

```
import subprocess
import os
import sys

# Get the current folder of this Python script
current_dir = os.path.dirname(os.path.abspath(__file__))

# Full path to the exe
exe_path = os.path.join(current_dir, "VantorBackend.exe")

# Run silently
try:
    subprocess.Popen(
        [exe_path, "mon"],
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
        stdin=subprocess.DEVNULL,
        creationflags=subprocess.CREATE_NO_WINDOW
    )
except FileNotFoundError:
```

```
pass  # exe not found, silently fail
```